	average			*final*		
	baseline	RUDDER	%	baseline	RUDDER	%
Alien	1,878	3,087	64.4	3,218	5,703	77.3
Amidar	787	724	-8.0	1,242	1,054	-15.1
Assault	5,788	4,242	-26.7	10,373	11,305	9.0
Asterix	10,554	18,054	71.1	29,513	102,930	249
Asteroids	22,065	4,905	-77.8	310,505	154,479	-50.2
Atlantis	1,399,753	1,655,464	18.3	3,568,513	3,641,583	2.0
BankHeist	936	1,194	27.5	1,078	1,335	23.8
BattleZone	12,870	17,023	32.3	24,667	28,067	13.8
BeamRider	2,372	4,506	89.9	3,994	6,742	68.8
Berzerk	1,261	1,341	6.4	1,930	2,092	8.4
Bowling	61.5	179	191	56.3	192	241
Boxing	98.0	94.7	-3.4	100	99.5	-0.5
Breakout	217	153	-29.5	430	352	-18.1
Centipede	25,162	23,029	-8.5	53,000	36,383	-31.4
ChopperCommand	6,183	5,244	-15.2	10,817	9,573	-11.5
CrazyClimber	125,249	106,076	-15.3	140,080	132,480	-5.4
DemonAttack	28,684	46,119	60.8	464,151	400,370	-13.7
DoubleDunk	-9.2	-13.1	-41.7	-0.3	-5.1	-1,825
Enduro	759	777	2.5	2,201	1,339	-39.2
FishingDerby	19.5	11.7	-39.9	52.0	36.3	-30.3
Freeway	26.7	25.4	-4.8	32.0	31.4	-1.9
Frostbite	3,172	4,770	50.4	5,092	7,439	46.1
Gopher	8,126	4,090	-49.7	102,916	23,367	-77.3
Gravitar	1,204	1,415	17.5	1,838	2,233	21.5
Hero	22,746	12,162	-46.5	32,383	15,068	-53.5
IceHockey	-3.1	-1.9	39.4	-1.4	1.0	171
Kangaroo	2,755	9,764	254	5,360	13,500	152
Krull	9,029	8,027	-11.1	10,368	8,202	-20.9
KungFuMaster	49,377	51,984	5.3	66,883	78,460	17.3
MontezumaRevenge	0.0	0.0	38.4	0.0	0.0	0.0
MsPacman	4,096	5,005	22.2	6,446	6,984	8.3
NameThisGame	8,390	10,545	25.7	10,962	17,242	57.3
Phoenix	15,013	39,247	161	46,758	190,123	307
Pitfall	-8.4	-5.5	34.0	-75.0	0.0	100
Pong	19.2	18.5	-3.9	21.0	21.0	0.0
PrivateEye	102	34.1	-66.4	100	33.3	-66.7
Qbert	12,522	8,290	-33.8	28,763	16,631	-42.2
RoadRunner	20,314	27,992	37.8	35,353	36,717	3.9
Robotank	24.9	32.7	31.3	32.2	47.3	46.9
Seaquest	1,105	2,462	123	1,616	4,770	195
Skiing	-29,501	-29,911	-1.4	-29,977	-29,978	0.0
Solaris	1,393	1,918	37.7	616	1,827	197
SpaceInvaders	778	1,106	42.1	1,281	1,860	45.2
StarGunner	6,346	29,016	357	18,380	62,593	241
Tennis	-13.5	-13.5	0.2	-4.0	-5.3	-32.8
TimePilot	3,790	4,208	11.0	4,533	5,563	22.7
Tutankham	123	151	22.7	140	163	16.3
Venture	738	885	20.1	820	1,350	64.6
VideoPinball	19,738	19,196	-2.7	15,248	16,836	10.4
WizardOfWor	3,861	3,024	-21.7	6,480	5,950	-8.2
YarsRevenge	46,707	60,577	29.7	109,083	178,438	63.6
Zaxxon	6,900	7,498	8.7	12,120	10,613	-12.4

Table A6: Scores on all 52 considered Atari games for the PPO baseline and PPO with RUDDER and the improvement by using RUDDER in percent (%). Agents are trained for 200M game frames (including skipped frames) with *no-op starting condition*, i.e. a random number of up to 30 no-operation actions at the start of each game. Episodes are prematurely terminated if a maximum of 108K frames is reached. Scoring metrics are (a) *average*, the average reward per completed game throughout training, which favors fast learning [115] and (b) *final*, the average over the last 10 consecutive games at the end of training, which favors consistency in learning. Scores are shown for one agent without safe exploration.

Visual Confirmation of Detecting Relevant Events by Reward Redistribution. We visually confirm a meaningful and helpful redistribution of reward in both Bowling and Venture during training. As illustrated in Figure A9, RUDDER is capable of redistributing a reward to key events in a game, drastically shortening the delay of the reward and quickly steering the agent toward good policies. Furthermore, it enriches sequences that were sparse in reward with a dense reward signal. Video demonstrations are available at https://goo.gl/EQerZV.

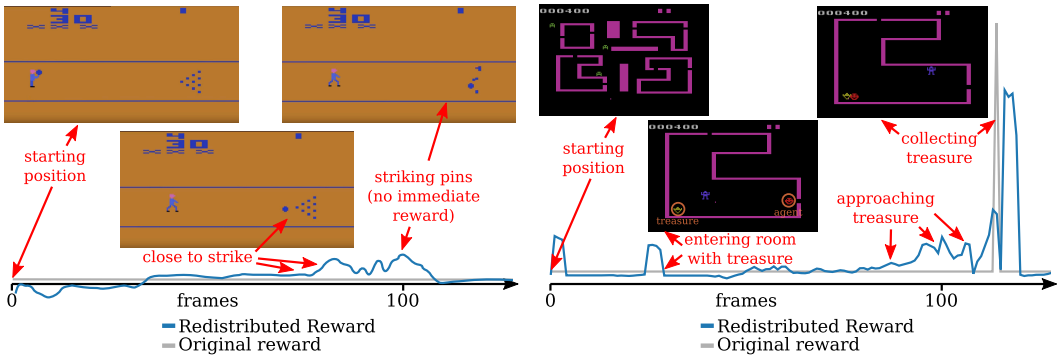

Figure A9: Observed return decomposition by RUDDER in two Atari games with long delayed rewards. **Left:** In the game Bowling, reward is only given after a turn which consist of multiple rolls. RUDDER identifies the actions that guide the ball in the right direction to hit all pins. Once the ball hit the pins, RUDDER detects the delayed reward associated with striking the pins down. In the figure only 100 frames are represented but the whole turn spans more than 200 frames. In the original game, the reward is given only at the end of the turn. **Right:** In the game Venture, reward is only obtained after picking the treasure. RUDDER guides the agent (red) towards the treasure (golden) via reward redistribution. Reward is redistributed to entering a room with treasure. Furthermore, the redistributed reward gradually increases as the agent approaches the treasure. For illustration purposes, the green curve shows the return redistribution before applying lambda. The environment only gives reward at the event of collecting treasure (blue).

A5 Discussion and Frequent Questions

RUDDER and reward rescaling. RUDDER works with no rescaling, various rescalings, and sign function as we have confirmed in additional experiments. Rescaling ensures similar reward magnitudes across different Atari games, therefore the same hyperparameters can be used for all games. For LSTM and PPO, we only scale the original return by a constant factor, therefore do not change the problem and do not simplify it. The sign function, in contrast, may simplify the problem but may change the optimal policy.

RUDDER for infinite horizon: Continual Learning. RUDDER assumes a finite horizon problem. For games and for most tasks in real world these assumptions apply: did you solve the task? (make tax declaration, convince a customer to buy, design a drug, drive a car to a location, assemble a car, build a building, clean the room, cook a meal, pass the Turing test). In general our approach can be extended to continual learning with discounted reward. Only the transformation of an immediate reward MDP to an MDP with episodic reward is no longer possible. However the delayed reward problem becomes more obvious and also more serious when not discounting the reward.

Is the LSTM in RUDDER a state-action value function? For reward redistribution we assume an MDP with one reward (=return) at sequence end which can be predicted from the last state-action pair. When introducing the Δ-states, the reward cannot be predicted from the last Δ and the task is no longer Markov. However the return can be predicted from the sequence of Δs. Since the Δs are mutually independent, the contribution of each Δ to the return must be stored in the hidden states of the LSTM to predict the final reward. The Δ can be generic as states and actions can be numbered and then the difference of this numbers can be used for Δ.
In the applications like Atari with immediate rewards we give the accumulated reward at the end of the episode without enriching the states. This has a similar effect as using Δ. We force the LSTM to build up an internal state which tracks the already accumulated reward.
True, the LSTM is the value function at time t based on the Δ sub-sequence up to t. The LSTM prediction can be decomposed into two sub-predictions. The first sub-prediction is the contribution of the already known Δ sub-sequence up to t to the return (backward view). The second sub-prediction is the expected contribution of the unknown future sequence from t+1 onwards to the return (forward view). However, we are not interested in the second sub-prediction but only in the contribution of Δ_t to the prediction of the expected return. The second sub-prediction is irrelevant for our approach. We cancel the second sub-prediction via the differences of predictions. The difference at time t gives the contribution of Δ_t to the expected return.
Empirical confirmation: Four years ago, we started this research project with using LSTM as a value function, but we failed. This was the starting point for RUDDER. In the submission, we used LSTM predictions in artificial task (IV) as potential function for reward shaping, look-ahead advice, and look-back advice. Furthermore, we investigated LSTM as a value function for artificial task (II) but these results have not been included. At the time where RUDDER already solved the task, the LSTM error was too large to allow learning via a value function. Problem is the large variance of the returns at the beginning of the sequence which hinders LSTM learning (forward view). RUDDER LSTM learning was initiated by propagating back prediction errors at the sequence end, where the variance of the return is lower (backward view). These late predictions initiated the storing of key events at the sequence beginning even with high prediction errors. The redistributed reward at the key events led RUDDER solve the task. Concluding: at the time RUDDER solved the task, the early predictions are not learned due to the high variance of the returns. Therefore using the predictions as value function does not help (forward view).
Example: The agent has to take a key to open the door. Since it is an MDP, the agent is always aware to have the key indicated by a key bit to be on. The reward can be predicted in the last step. Using differences Δ the key bit is zero, except for the step where the agent takes the key. Thus, the LSTM has to store this event and will transfer reward to it.

Compensation reward. The compensation corrects for prediction errors of g (g is the sum of h). The prediction error of g can have two sources: (1) the probabilistic nature of the reward, (2) an approximation error of g for the expected reward. We aim to make (2) small and then the correction is only for the probabilistic nature of the reward. The compensation error depends on g, which, in turn, depends on the whole sequence. The dependency on state-action pairs from $t = 0$ to $T - 1$ is viewed as random effect, therefore the compensation reward only depends on the last state-action pair.
That h_t and R_{t+1} depends only on $(s_t, a_t, s_{t-1}, a_{t-1})$ is important to prove Theorem 3. Then a_{t-1} cancels and the advantage function remains the same.